Table of Contents

Introduction 2
Problem Definition 2
Data Sources 2
Aims And Objectives 3
Aim 3
Objectives 3
Data Preparation 4
Exploratory Data Analysis 5
Exploratory Data Analysis - Conclusion 8
Classification Model Testing 9
Models selected 9
Gaussian Naive Bayes Results 9
PCA Analysis 10
Random Forest Classification Model 11
GridSearchCV for Hyperparameter Tuning 12
KNN Classification Model 12
Cross Validation 14
Conclusion 15
Ethical and Legal Considerations 16
References 17
Appendix 18

Introduction

Machine Learning techniques are widely used for predictive decision making across many industries. This assignment investigates whether games in the English soccer Premier League, will have over 2.5 goals per game. The Over 2.5 goals is a common benchmark in football analytics and betting markets.

Problem Definition

Today, many football clubs, sports betting companies and sports media corporations have employed data experts to analyse in-play statistics to help improve team performance. The challenge is determining whether machine learning can improve prediction accuracy through the analysis of large-scale historical football datasets.

Data Sources

The primary data source used for this project will be:

https://football-data.co.uk/data.php

This is a publicly available repository containing the required historical match statistics.

These variables will be relevant to the proposed business problem as they provide the information required for machine learning models to identify patterns associated with predicting football match outcomes. The data will be used for educational and research purposes only.

Aims And Objectives

Aim

The aim of this project is to develop and evaluate machine learning classification models capable of predicting whether an English Premier League match would finish with over 2.5 goals.

Objectives

The objectives were to prepare and integrate six seasons of data, identify predictive features, compare model performance across multiple train-tests splits and evaluate the impact of dimensionality reduction using PCA.

The Models used for this project include:

Gaussian Naïve Bayes Model 5%, 15% and 25% Test Split cases

PCA Analysis using 25% Test Split case

Random Forest Classification Model 25% Test Split case with Confusion Matrix

KNN Model 25% Test Split case with Confusion Matrix, with neighbours set as 10

Cross-Validation

GridSearchCV

Data Preparation

The goal at Data Preparation stage was to import the 6 csv files for the six Premier League years from 2020/2021 to 2025/26, into a Data Frame:

These variables were relevant to the proposed business problem as they provide the information required for machine learning models to identify patterns associated with predicting football match outcomes. Relevant match statistics including goals scored, shots on target and corners were taken from the original dataset.

A Python function was developed to import the Premier League data, select the relevant data and create feature engineering to add additional features including half time goals, full-time goals, total shots on target, to support the machine learning analysis.

Once the data was collected, I carried out analysis to check for:

- Missing Data
- Duplicate Data
- Football Teams naming conventions, ie Man Utd or Manchester United

Once the data has been cleaned, I merged all six Data Frames into 1 large Data Frame. The master Data Frame was named “df_PL_Master”.

Exploratory Data Analysis

At the EDA stage, I created analysis of the master file. The Target Variable was created, called "Over_2.5", where 3 or more total goals = 1, else = 0. There is no requirement to remove outliers, as the Target is simply asking if the game ended in over 2.5 goals or not.

Target Variable Distribution

The Target Variable will be Over 2.5 goals

Find the results of over 2.5

```
43]: df_PL_Master["Over_2.5"] = np.where(df_PL_Master["FT_Goals"] > 2, 1,0)

     print(df_PL_Master["Over_2.5"].value_counts() )
```

```
Over_2.5
1    1265
0    1015
Name: count, dtype: int64
```

Show the over 2.5 Goals as a Bar Chart

```
44]: sns.countplot(
         data=df_PL_Master,
         x="Over_2.5"
     )
```

```
44]: <Axes: xlabel='Over_2.5', ylabel='count'>
```

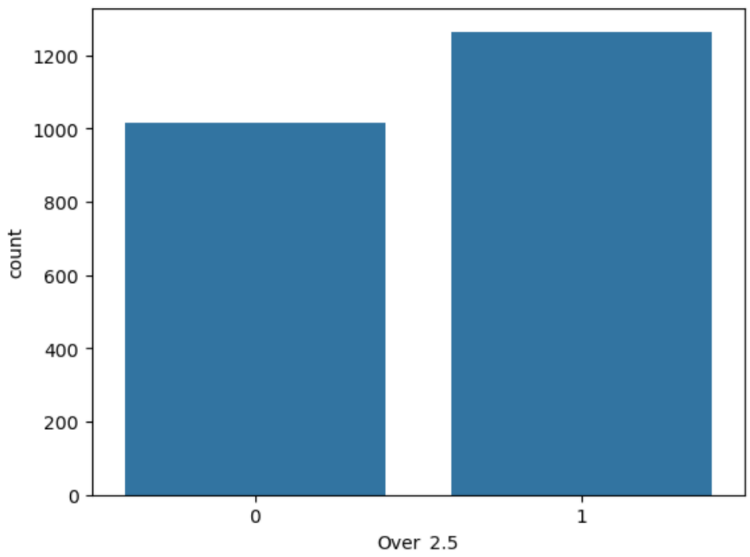
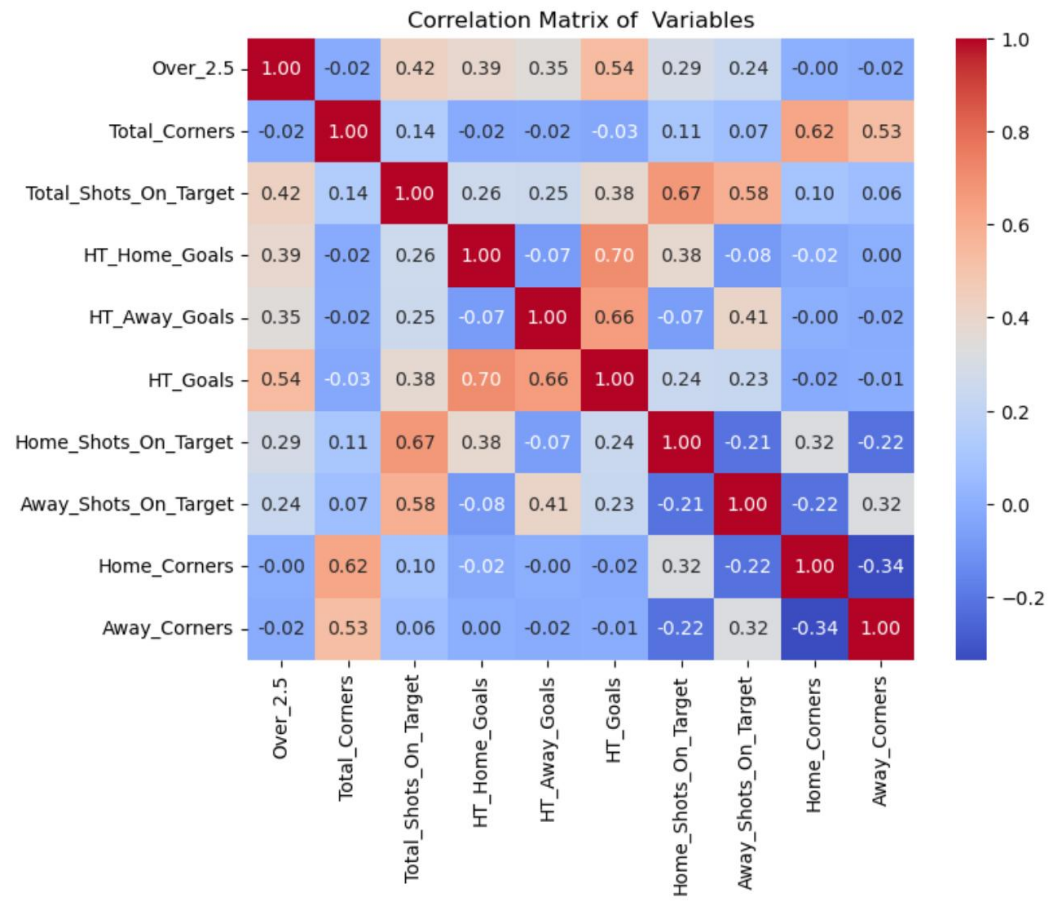


Figure 1 Shows the creation of the Target variable

A correlation heatmap was created to find correlation with Over 2.5 goals.

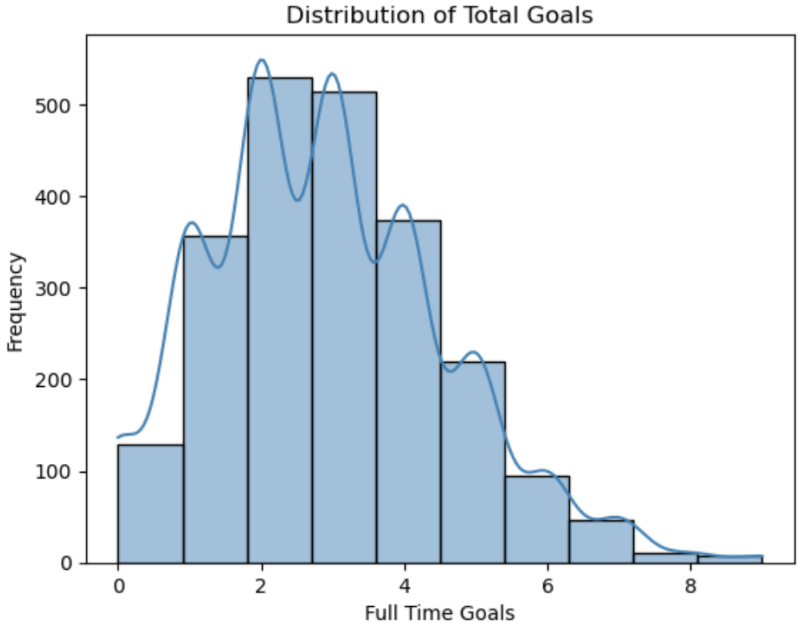


The above heatmap shows strong correlation to Over 2.5 Goals from:

Half Time Goals 0.54
Half Time Home Goals 0.39
Half Time Away Goals 0.35
Total Shots on Target 0.42

Figure 2 Shows a correlation heatmap with over 2.5 goals

Total Goals analysis showed a positive skew. A Shapiro-Wilk test was also carried out to show that the sample did not appear to be normally distributed.



The distribution of total goals is positively skewed

Most matches produce 1-4 goals per game

Matches with more than 6 goals are relatively rare

The histogram demonstrates that the target variable of over 2.5 goals is appropriate for classification

Check Normality (using Shapiro_Wilk_test)

[89]:
```
from scipy import stats
from scipy.stats import norm # for normality test
```

[90]:
```
stat, p_value = stats.shapiro(df_PL_Master["FT_Goals"])

print(stat,p_value)
```

```
0.9495699788214029 2.6349078228629415e-27
```

[91]:
```
if p_value >= 0.05:
    print("Sample appears to be normally distributed.")
else:
    print("Sample does not appear to be normally distributed.")
```

```
Sample does not appear to be normally distributed.
```

Figure 3 Shows the Distribution of Total Goals

Exploratory Data Analysis - Conclusion

The analysis has identified that half time goals along with shots on target have a higher correlation to games ending in more than 2.5 goals. In contrast the corners statistics showed little correlation, indicating that attacking action seems to be a more useful predictor overall

Classification Model Testing

Models selected

Three Classification models will be selected for this project.

Gaussian Naive Bayes, Random Forest and K-Nearest Neighbours were the classification models selected to compare probabilistic, ensemble and distance-based approaches.

To run the classification model, a Target variable was created earlier as a binary classification, where 3 or more goals =1, else =0. The dependent Y Target is "Over 2.5" column

The train-test configurations consisted of 5%, 15% and 25% test datasets. Stratified sampling was applied (stratify=y).

Gaussian Naive Bayes Results

The Gaussian Naive Bayes model achieved an accuracy of:

1. 5% Test split = 81.58%
2. 15% Test split = 79.24%
3. 25% Test split = 76.49%

The results remained relatively consistent, and the model performed well across unseen data, suggesting that it did not appear to suffer from any substantial overfitting.

```
7]: print(classification_report(
        y_test,
        y_pred_nb)
    )
```

	precision	recall	f1-score	support
0	0.70	0.83	0.76	254
1	0.84	0.71	0.77	316
accuracy			0.76	570
macro avg	0.77	0.77	0.76	570
weighted avg	0.78	0.76	0.77	570

Figure 4 shows Gaussian Naive Bayes model at 25% Test split

PCA Analysis

Principal Component Analysis (PCA) was the applied dimensionality reduction technique.

PCA reduced the total columns from 9 to 5, whilst still retaining 94.63% of the total variance. Accuracy score was reduced slightly to 75.96%, suggesting that dimensionality reduction can still produce comparable predictive performance.

```
: print(classification_report(
      y_test,
      y_pred_pca)
  )
```

	precision	recall	f1-score	support
0	0.71	0.78	0.74	254
1	0.81	0.74	0.77	316
accuracy			0.76	570
macro avg	0.76	0.76	0.76	570
weighted avg	0.76	0.76	0.76	570

```
: # print(f"The number of columns in the original dataset was: {No_Columns} ")
  print(f"The number of numeric columns in the dataset, prior to PCA was: {num_features} ")
  print(f"The number of components retained after PCA was: {pca.n_components_} ")
  print(f"The percentage of Variance retained was: {pca.explained_variance_ratio_.sum()*100:.2f} %")
```

```
The number of numeric columns in the dataset, prior to PCA was: 9
The number of components retained after PCA was: 5
The percentage of Variance retained was: 94.63 %
```

Figure 5 shows PCA classification report

The precision results for over 2.5 goals showed the highest result, with 81% accuracy.

Random Forest Classification Model

The Random Forest classifier achieved an accuracy of 74.74%. The model achieved recall score of 78% for over 2.5 goals.

```
# Print Accuracy rounded upto 2 decimal places
rf_score = accuracy_score(
    y_test,
    y_pred_rf)


print(f"The Random Forest Accuracy score is : {rf_score:.2%} ")
```

```
The Random Forest Accuracy score is : 74.74%
```

```
print(classification_report(
    y_test,
    y_pred_rf)
)
```

```
              precision    recall  f1-score   support

           0       0.72      0.71      0.71       254
           1       0.77      0.78      0.77       316

    accuracy                           0.75       570
   macro avg       0.74      0.74      0.74       570
weighted avg       0.75      0.75      0.75       570
```

Figure 6 Shows the classification report for the Random Forest Classifier

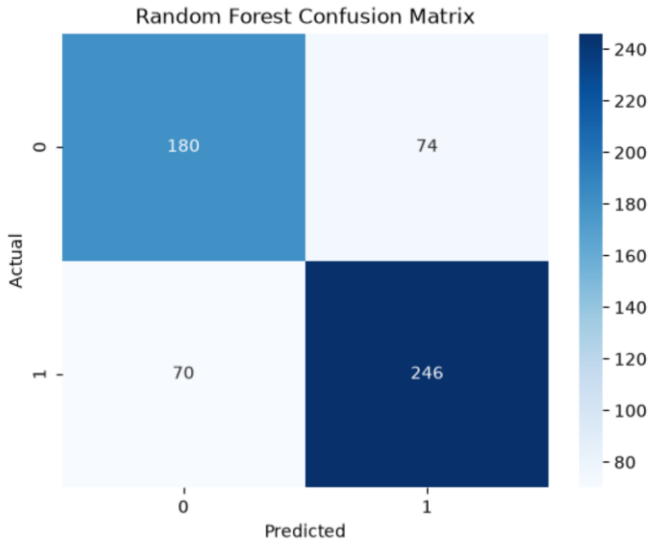


Figure 7 shows the Random Forest Confusion Matrix

GridSearchCV for Hyperparameter Tuning

GridSearchCV was applied to the Random Forest Classifier and the optimal parameter settings identified were:

- n_estimators = 100
- max_depth = 10
- min_samples_split = 5

The model achieved a mean Cross Validation Accuracy of 76.67%, demonstrating that modest improvements in predictive performance can be achieved through hyperparameter tuning.

KNN Classification Model

The KNN classifier achieved an accuracy with k = 10 of 71.40%. This was the weakest of the three models.

```
# Print Accuracy rounded upto 2 decimal places
knn_score = accuracy_score(
    y_test,
    y_pred_knn)


print(f"The KNN Accuracy score is : {knn_score:.2%} ")
```

```
The KNN Accuracy score is : 71.40%
```

```
# Display the classification report
print(classification_report(y_test, y_predict))
```

```
              precision    recall  f1-score   support

           0       0.66      0.73      0.69       254
           1       0.76      0.70      0.73       316

    accuracy                           0.71       570
   macro avg       0.71      0.72      0.71       570
weighted avg       0.72      0.71      0.71       570
```

Figure 8 Shows the KNN classification report

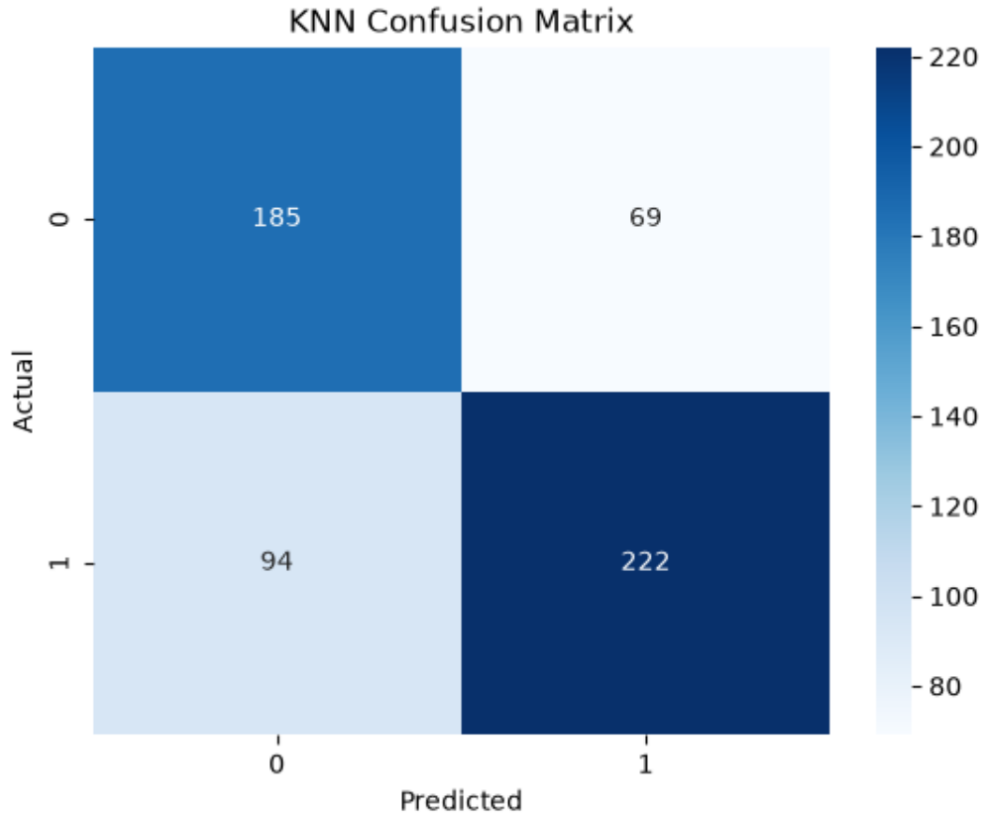


Figure 9 Shows the KNN Confusion Matrix

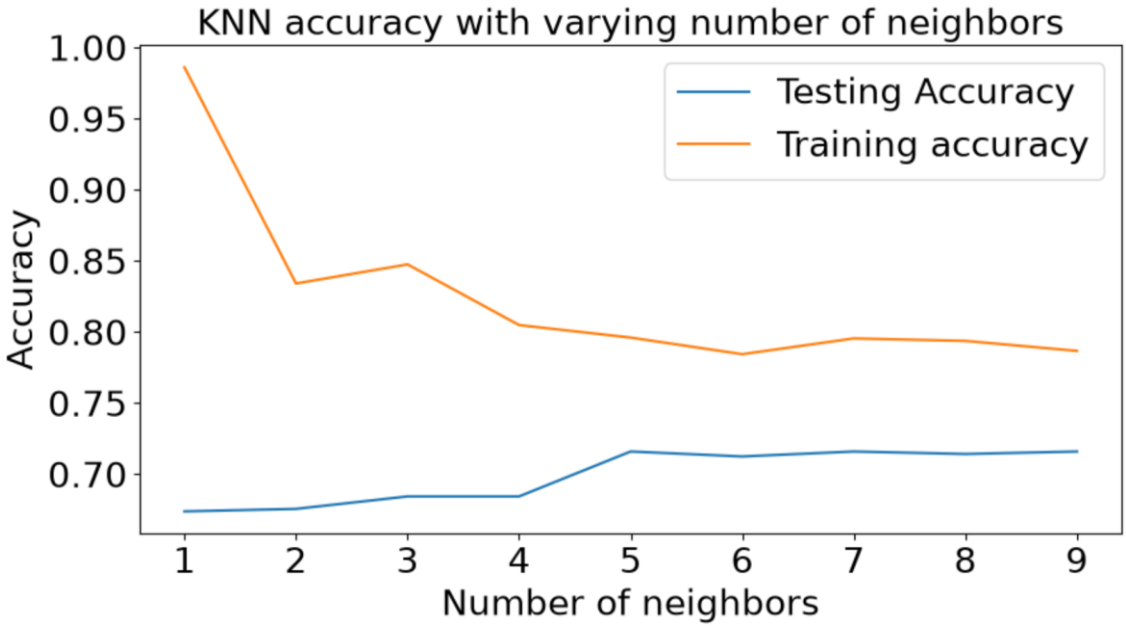


Figure 10 Shows a KNN graph of the Testing & Training Accuracy

Cross Validation

Cross Validation was carried out for the 3 models, running the 25% Test Split

1. Gaussian
2. Random Forest
3. KNN

```
# Convert results to DataFrame for easy viewing
metrics_df = pd.DataFrame(results, columns=["Model", "Accuracy", "Precision", "Recall", "F1 Score"])
print("Model Performance Metrics on the 25% Test Split :")
print(metrics_df)
```

```
Model Performance Metrics on the 25% Test Split :
                 Model  Accuracy  Precision    Recall  F1 Score
0   k-Nearest Neighbors  0.705263   0.705747  0.705263  0.705427
1         Random Forest  0.759649   0.759495  0.759649  0.759174
2  Gaussian Naive Bayes  0.777193   0.778702  0.777193  0.777409
```

```
# Cross-validation for accuracy on the full dataset
cv_results = []
for name, classifier in classifiers.items():
    cv_scores = cross_val_score(classifier, X, y, cv=5, scoring='accuracy')
    cv_results.append([name, cv_scores.mean()])

# Display cross-validation results
cv_results_df = pd.DataFrame(cv_results, columns=["Model", "Mean CV Accuracy"])
print("\nCross-Validation Accuracy Scores:")
print(cv_results_df)
```

```
Cross-Validation Accuracy Scores:
                 Model  Mean CV Accuracy
0   k-Nearest Neighbors          0.723246
1         Random Forest          0.747368
2  Gaussian Naive Bayes          0.762719
```

Figure 11Shows the Cross Validation Accuracy scores

Gaussian Naive Bayes achieved the highest mean cross validation score of 76.27%. The results show a consistent performance between the model test set results and cross validation results, suggesting that any overfitting was not observed.

Conclusion

The results suggest that half time goals and shots on target are the most influential predictors of matches finishing over 2.5 goals. Three classification techniques were evaluated along with PCA based dimensionality reduction.

The Gaussian Naive Bayes classifier achieved the best performance with 76.49% accuracy, using the 25% Test Split.

PCA also demonstrated that dimensionality reduction can maintain comparable prediction performance, after reducing the number of columns from 9 to 5.

Random Forest achieved comparable results with hyperparameter tuning introduced.

KNN displayed the weakest results in predictive performance.

Overall, the results show that machine learning techniques can be successfully applied to predict over 2.5 goals in the Premier League.

Word count 997

Ethical and Legal Considerations

The dataset contains only historical football match information and does not contain personal, confidential or sensitive data and any external sources and supporting materials will be appropriately referenced using Harvard Referencing standards.

The use of Artificial intelligence tools will be documented in accordance with the AI Fluency Framework, specified by CCT College

References

Football-Data.co.uk (2026) Football Results and Betting Data. Available at: https://football-data.co.uk/data (accessed: 1 October 2026)

CCT College Dublin (2026) Data Preparation Module Notes. Available via Moodle (Accessed: 10 October 2026)

CCT College Dublin (2026) Statistical Techniques Module Notes. Available via Moodle (Accessed: 24 October 2026)

CCT College Dublin (2026) Machine Learning Module Notes. Available via Moodle (Accessed: 1 October 2026)

Python for Machine Learning & Data Science Masterclass by Pierian Data. Available at: www.udemy.com (Accessed: 15 October 2026)

Microsoft (2026) Microsoft Copilot. Available at https://copilot.Microsoft.com (Accessed: 1st November 2026)

Datacamp.com (2026) Available at: https://www.datacamp.com/tutorial/random-forests-classifier-python (Accessed 04/11/2026)

Pandas.org (2026) Available at: https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html (Accessed 22/10/2026)

Appendix

Appendix A – Reflection on the use of Artificial Intelligence

Artificial Intelligence (Microsoft 365 Copilot) was used throughout this project as a support tool to assist with understanding Python programming and report planning.

AI was also used to provide guidance when debugging code errors in Python, such as helping with running the test split cases for classification modelling. In addition, AI provided suggestions on how correctly run the PCA model and provided help with coding of the Confusion Models, GridSearchCV and Cross-Validation.

AI also helped with my report writing, mainly reducing the word count, helping to make it more streamlined. It also highlighted issued still to be covered in the brief, such as model tuning, although at this stage, I had not completed my research.

The final decisions regarding the choice of modelling techniques, interpretation of results and conclusions remained my own responsibility.

Overall, the use of AI enhanced my learning experience by acting as a study aid and technical assistant, improving both my understanding of modelling techniques and my confidence in applying Python for data analysis.

Appendix A

Conversations with AI

Appendix A – 1

02/11/2026 – Advice on Test Splitting

CA1_Machine_Learnin...

Hi see attached MAchine Learning for the last 6 premier leagues. Is this being done in the correct format for a 5%, 15% and 25% test.

Hi Ciaran,

I've reviewed your Python file, CA1_Machine_Learning (1).py. live

Short Answer

Yes, your 5%, 15%, and 25% train/test splits are being implemented correctly. You are:

1. Creating X and y once.
2. Using `train_test_split()`.
3. Training a new model for each test size.
4. Scaling the training data and applying the same transformation to the test data.
5. Evaluating using Accuracy and Classification Report.

That is exactly what I would expect for comparing different test split percentages. live

---

One improvement I would make

Because this is a classification problem, I would add **stratification**:

Python

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.05,
    random_state=82,
    stratify=y
)
```

and similarly for the 15% and 25% splits.

This ensures the proportion of:

- Over 2.5 = 1
- Under 2.5 = 0

remains consistent in both training and testing datasets.

More important issue: PCA section is not actually using PCA

This is the main problem I spotted.

You correctly create:

Python

```python
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=0.90)
X_pca = pca.fit_transform(X_scaled)
```

However later you split:

Python

```python
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=82
)
```

Notice you're using **X**, not **X_pca**.

Therefore the model in the PCA section is actually being trained on the original dataset again.

So your PCA accuracy:

Python

```python
print(f"The PCA Accuray Score is : {score:.2%}")
```

is not a PCA result.

Correct PCA approach

Instead do:

Python

```python
X_train, X_test, y_train, y_test = train_test_split(
    X_pca,
    y,
    test_size=0.25,
    random_state=82,
    stratify=y
)
```

Then:

Python

```python
nb_model = GaussianNB()

nb_model.fit(X_train, y_train)

y_pred = nb_model.predict(X_test)

score = accuracy_score(y_test, y_pred)
```

No additional scaling is required because PCA has already been performed on the scaled data.

Appendix A – 2

02/11/2026 Help with report writing

CA1 - Machine Learnin... CA1 without references

see the attached brief and also my report. at present i am at 1367 words. i need to reduce this to approx 1000 words

Ciaran, I've reviewed both the brief and your report.

The good news is that **you are not actually 367 words over the limit in practical terms**.

The brief specifically states that **headings, diagrams, figures, code, references and citations are excluded from the word count**.

Looking at your report, a large amount of space is taken by:

- Figure 1
- Figure 2
- Figure 3
- Headings
- Bullet lists
- Dataset field listings

These generally would not count as narrative report words.

Bigger Issue: Missing Assignment Requirements

Before worrying too much about word count, I would be more concerned that the brief requires:

> "Develop at least two machine learning models"

and

> "Apply specific techniques (e.g. GridSearchCV) to tune the ML models"

and

> "Use cross-validation methods"

Your report currently discusses:

- Gaussian Naive Bayes
- PCA

But PCA is not a second machine learning model. It is a dimensionality reduction technique.

If your notebook only contains Naive Bayes, I would expect marks to be lost in Criteria 3.

Where You Can Easily Save 250-300 Words

1. Reduce the Introduction

Currently:

> Machine Learning is a widely used tool helping to support predictive decision making...

...through to the end of the explanation of why 2.5 goals was chosen.

You could replace all of that with:

> Machine learning techniques are widely used for predictive decision-making across many industries. This project investigates whether historical English Premier League statistics can be used to predict whether a match will finish with over 2.5 goals. The over 2.5 threshold was selected because it is a commonly used benchmark in football analytics and betting markets.

That saves approximately **80 words**.

---

2. Reduce the Aims and Objectives

You currently have:

- Aim
- Long Aim paragraph
- Five objectives

The lecturer already knows the assignment requirements.

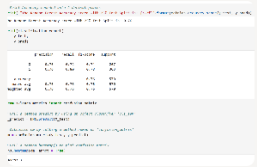
Replace with:

> The aim of this project was to develop and evaluate machine learning models capable of predicting whether an English Premier League match would finish with over 2.5 goals. The objectives were to prepare and integrate six seasons of data, identify important predictive features, compare model performance across multiple train-test splits and evaluate the impact of dimensionality reduction using PCA.

Saves about **120 words**.

Appendix A – 3

05/11/2026 – Help with confusion matrix coding



i am running a confusion matrix for my random forest model, i have taken the code from the KNN model, how do i implement it correctly for random forest? # Call a method predict by using an object classifier 'cls_svm'
y_predict = kNN.predict(X_test)

For your **Random Forest** model, you simply replace the KNN model object with your Random Forest model object.

You've already generated predictions earlier:

Python

```
y_pred = rf_model.predict(X_test)
```

(or whatever you called your Random Forest model).

So for the confusion matrix, use the Random Forest model instead of kNN.

Option 1: Reuse y_pred

Since you've already created predictions:

Python

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest Confusion Matrix")

plt.show()
```

Appendix A – 4

05/11/2026 – error with confusion matrix code



can you see an error here

Yes 👍, the error is on the last line:

Python

```
1  plt("Show")
```

`plt` is the matplotlib module, not a function, so Python is saying:

> TypeError: 'module' object is not callable

Change this:

Python

```
1  plt("Show")
```

To this:

Python

```
1  plt.show()
```

Your completed code should be:

Your completed code should be:

Python

```python
from sklearn.metrics import confusion_matrix

y_predict = clf_RF.predict(X_test)

cm = confusion_matrix(y_test, y_predict)

sns.heatmap(cm, annot=True)

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

Another improvement

Your heatmap is displaying scientific notation (`1.9e+02`).

To show whole numbers instead:

Python

```python
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

So I'd use:

Python

```python
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues')

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

That will give a much cleaner confusion matrix for your report. 🎯

The fact that the matrix appeared underneath the error also confirms everything else worked correctly. The only problem was `plt("Show")`.

Appendix A – 5

09/11/2026 – helping on errors with GridSearchCV

i have set up a GridSearch for RandomForest and KNN, but t seems to have stalled out

I can see why it feels stalled. Looking at your GridSearch code, you've actually created a **fairly large search space**. live

For Random Forest you have:

Python

```
"n_estimators": [50, 100, 200],      # 3
"max_depth": [None, 10, 20, 30],     # 4
"min_samples_split": [2, 5, 10],     # 3
"min_samples_leaf": [1, 2, 4]        # 3
```

Total combinations:

Python

```
3 × 4 × 3 × 3 = 108
```

Then you have:

Python

```
cv=5
```

so:

Python

```
108 × 5 = 540 model fits
```

for Random Forest alone. live

Then KNN:

Python

```
"n_neighbors": [3, 5, 7, 9],         # 4
"algorithm": ['auto','ball_tree',
              'kd_tree','brute']     # 4
```

gives:

```python
4 × 4 = 16
```

and

```python
16 × 5 = 80 fits
```

Total:

```python
540 + 80 = 620 model fits
```

on a dataset of ~2,280 matches. live

That can take quite a while.

For the assignment I'd simplify

The brief only requires you to demonstrate GridSearchCV.

I would reduce Random Forest to:

```python
"Random Forest": {
    "n_estimators": [100, 200],
    "max_depth": [10, 20],
    "min_samples_split": [2, 5]
}
```

Now:

```python
2 × 2 × 2 = 8 combinations
```

and

```python
8 × 5 = 40 fits
```

instead of 540.

KNN

Likewise:

```python
"k-Nearest Neighbors": {
    "n_neighbors": [3,5,7,9]
}
```

Only:

```python
4 × 5 = 20 fits
```

Also add this

Change:

```python
verbose = 2
```

to:

```python
verbose = 3
```

Appendix A – 6

09/11/2026 – helping with more issues using GridSearchCV



can you spot the error

Yep 😀, and the good news is the GridSearch actually worked.

This line succeeded:

Python

```
print(rf_grid.best_params_)
```

and returned:

Python

```
{'max_depth': 10,
 'min_samples_split': 5,
 'n_estimators': 200}
```

So GridSearch completed successfully.

The error is on the next line:

Python

```
print(rf.grid.best_score_)
```

You have:

Python

```
rf.grid
```

but there is no attribute called `grid` on a `RandomForestClassifier`.

Correct code

Python

```python
print(rf_grid.best_params_)

print(rf_grid.best_score_)
```

You can also get the best model itself:

Python

```python
best_rf = rf_grid.best_estimator_

print(best_rf)
```

Your Random Forest tuning results

You already have the important result:

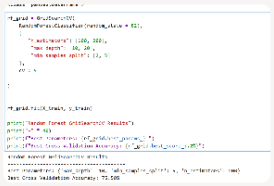
Python

```
Best Parameters

max_depth = 10
min_samples_split = 5
n_estimators = 200
```

Appendix A – 7

09/11/2026 – helping with Cross-Validation reporting



looks better, should i put this Cross validation in with the Random Forest Testing

Yes, I would.

If I were marking the assignment, I'd expect to see the flow:

Random Forest Model

- Why Random Forest was chosen
- 25% Train/Test Split results
- Classification Report
- Confusion Matrix
- Cross Validation
- GridSearchCV / Hyperparameter Tuning
- Discussion

rather than having Cross Validation separated from the model.

Something like:

Random Forest Classification Model

Random Forest was selected due to its ability to identify non-linear relationships between football match statistics and the target variable. The model combines multiple decision trees, reducing the risk of overfitting while improving predictive performance.

Appendix A – 8

09/11/2026 – helping with Report layout and requirements covered.

CA1_Machine_Learning

See the layout now. Does this look to have covered all the necessary requirements and is the layout in the correct flow

I've gone through the latest version and overall I think you're in a very good position now. The flow is logical and, more importantly, it maps well to the assignment requirements. live

Overall Structure

Your flow is now:

1. Data Preparation
2. EDA
3. Gaussian Naive Bayes
   - 5%, 15%, 25% test splits
   - Classification Reports
   - PCA
4. Random Forest
   - Accuracy
   - Classification Report
   - Confusion Matrix
   - GridSearchCV
5. KNN
   - Accuracy
   - Classification Report
   - Confusion Matrix
   - Neighbour comparison chart
6. Cross Validation Comparison
7. Conclusion
8. References

That's a sensible machine learning workflow. live

Requirement Check

Requirement	Covered
Data Preparation	✅
Exploratory Data Analysis	✅
Multiple ML Models	✅
Model Comparison	✅
PCA	✅
Cross Validation	✅
GridSearchCV / Hyperparameter Tuning	✅
Confusion Matrices	✅
Overfitting Discussion	✅
Interpretation of Results	✅
Final Conclusion	✅

Honestly, I don't see any major missing requirement anymore. live